NATIONAL ECONOMICS UNIVERSITY

College of Technology

PROJECT 01

Library Management System

Subject: Introduction to Databases

Instructor: Mr. Tran Hung

Name: Bui Vi Anh - DS66B

Student ID: 11247122

Github: github.com/pipotht/Library-Management-System

Hanoi, 2026

Contents

1. Introduction

1.1 Project Overview

This report documents the design, implementation, and demonstration of a Library Management System built as the final project for the Introduction to Databases course at the National Economics University. The system uses MySQL as the relational database engine and Python as the application programming layer.

A library management system is a practical and well-defined domain for demonstrating core database concepts: normalized table design, referential integrity, transactional operations, and automated behavior via triggers and stored procedures. The project covers the full lifecycle from schema design to a working console application.

The source code is organized as follows:

- `database/finaldb.sql` – Full SQL script: schema, sample data, indexes, views, stored procedures, triggers, and UDF.
- `src/main.py` – Python CLI application.

1.2 Objectives

- Design a fully normalized relational database schema that covers books, authors, categories, readers, and borrowing transactions.
- Implement advanced database objects: indexes, views, stored procedures, triggers, and user-defined functions.
- Develop a Python CLI application that exposes all core functionalities through an interactive menu system.
- Enforce data integrity at the database level through constraints, foreign keys, and triggers.
- Apply basic database security and administration principles.

1.3 Technologies

Table 1.1: Technology Stack

Component	**Details**
DBMS	MySQL 8.x
Programming Language	Python 3.x
DB Connector	mysql-connector-python
Output Formatting	tabulate (fancy_grid)
SQL Script	`database/finaldb.sql`
Application	`src/main.py`
Development Tools	MySQL Workbench, VS Code, Terminal

2. System Analysis & Requirements

2.1 Functional Requirements

Book Management

- View the full book catalog with author and category details.
- Add new books with confirmation before inserting.
- Edit book quantity.
- Delete a book (with confirmation and FK protection).
- Search books by name using partial keyword matching (`LIKE`).

Reader Management

- View all registered readers.
- Register a new reader (name, address, phone).
- Update reader address and phone number.

Author & Category Management

- View, add, and delete authors.
- View, add, and delete categories.

Borrowing & Returning

- Borrow a book: validates reader and book existence, shows available quantity, calls stored procedure `BorrowBook`.
- Return a book: shows borrow details and days borrowed, warns if overdue, calls stored procedure `ReturnBook`.
- Prevents returning an already-returned record.

Reports

- Full borrowing report showing all transactions with status.
- Overdue report using UDF `fn_DaysOverdue` to calculate days borrowed and filter records exceeding 7 days.

- Statistical summary: total books, readers, and borrowing counts.
- Top 5 most borrowed books.

2.2 Non-Functional Requirements

Table 2.1: Non-Functional Requirements

Requirement	Implementation
Data Integrity	Foreign keys, CHECK constraints, NOT NULL, UNIQUE
Automation	Triggers auto-update book quantity on borrow/return
Performance	Indexes on `BookName` and `AuthorID` columns
Usability	Numbered CLI menu with confirmation prompts
Error Handling	try/except blocks, status validation before return

2.3 Entity Relationships

- **Categories → Books:** A category can classify many books (1:N).
- **Authors → Books:** An author can write many books (1:N).
- **Readers → Borrowing:** One reader can have many borrowing records (1:N).
- **Books → Borrowing:** One book can appear in many borrowing records (1:N).

2.4 Entity Relationship Diagram (ERD)

The ERD illustrates the logical structure of the database and ensures that all relationships are properly normalized and free from redundancy. Each entity is uniquely identified by a primary key, while foreign keys enforce referential integrity across related tables.

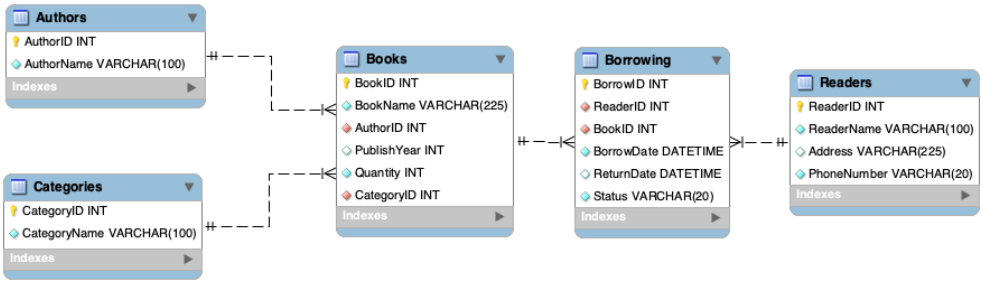


Figure 2.1: Entity Relationship Diagram of the Library Management System

3. Database Design & Implementation

3.1 Schema Overview

The database `LibraryDB` consists of five tables defined in `database/finaldb.sql`. All foreign keys use `ON DELETE RESTRICT` to prevent orphaned records, and `ON UPDATE CASCADE` to propagate ID changes automatically.

3.2 Table Structures

3.2.1 Categories

Table 3.1: Categories Table

Column	Type	Constraints
CategoryID	INT	PRIMARY KEY, AUTO_INCREMENT
CategoryName	VARCHAR(100)	NOT NULL, UNIQUE

3.2.2 Authors

Table 3.2: Authors Table

Column	Type	Constraints
AuthorID	INT	PRIMARY KEY, AUTO_INCREMENT
AuthorName	VARCHAR(100)	NOT NULL

3.2.3 Books

Table 3.3: Books Table

Column	Type	Constraints
BookID	INT	PRIMARY KEY, AUTO_INCREMENT
BookName	VARCHAR(255)	NOT NULL
AuthorID	INT	NOT NULL, FK → Authors(AuthorID)
PublishYear	INT	—
Quantity	INT	NOT NULL, DEFAULT 0, CHECK (≥ 0)
CategoryID	INT	NOT NULL, FK → Categories(CategoryID)

3.2.4 Readers

Table 3.4: Readers Table

Column	Type	Constraints
ReaderID	INT	PRIMARY KEY, AUTO_INCREMENT
ReaderName	VARCHAR(100)	NOT NULL
Address	VARCHAR(255)	—
PhoneNumber	VARCHAR(20)	NOT NULL, UNIQUE

3.2.5 Borrowing

Table 3.5: Borrowing Table

Column	Type	Constraints
BorrowID	INT	PRIMARY KEY, AUTO_INCREMENT
ReaderID	INT	NOT NULL, FK → Readers(ReaderID)
BookID	INT	NOT NULL, FK → Books(BookID)
BorrowDate	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP
ReturnDate	DATETIME	Nullable — set on return
Status	VARCHAR(20)	NOT NULL, DEFAULT 'Borrowed', CHECK IN ('Borrowed','Returned')

3.3 Sample Data

Table 3.6: Sample Data Summary

Table	Records	Notes
Categories	10	Novel, Science, History, Technology, Education, Psychology, Business, Art, Children, Philosophy
Authors	10	Mix of Vietnamese and international authors
Books	10	Titles spanning 1934–2011, quantities 5–20
Readers	10	Readers from cities across Vietnam
Borrowing	10	Mix of Borrowed and Returned; 3 records backdated to simulate overdue

Three borrowing records were manually backdated to simulate overdue scenarios:

```
UPDATE Borrowing SET BorrowDate = NOW() - INTERVAL 10 DAY WHERE
   BorrowID = 1;
UPDATE Borrowing SET BorrowDate = NOW() - INTERVAL 15 DAY WHERE
   BorrowID = 4;
UPDATE Borrowing SET BorrowDate = NOW() - INTERVAL  8 DAY WHERE
   BorrowID = 6;
```

4. Advanced Database Objects

4.1 Indexes

```
CREATE INDEX idx_bookname ON Books(BookName);
CREATE INDEX idx_author   ON Books(AuthorID);
```

`idx_bookname` accelerates `LIKE`-based keyword searches in the `search_book()` function. `idx_author` speeds up `JOIN` operations between `Books` and `Authors`, which appear in nearly every query that displays book details.

4.2 Views

4.2.1 View_BookDetails

```
CREATE VIEW View_BookDetails AS
SELECT b.BookID, b.BookName, a.AuthorName,
       c.CategoryName, b.Quantity
FROM Books b
JOIN Authors    a ON b.AuthorID   = a.AuthorID
JOIN Categories c ON b.CategoryID = c.CategoryID;
```

Listing 4.1: View_BookDetails

4.2.2 View_BorrowingReport

```
CREATE VIEW View_BorrowingReport AS
SELECT r.ReaderName, b.BookName,
       br.BorrowDate, br.ReturnDate, br.Status
FROM Borrowing br
JOIN Readers r ON br.ReaderID = r.ReaderID
JOIN Books   b ON br.BookID   = b.BookID;
```

Listing 4.2: View_BorrowingReport

4.2.3 View_OverdueReport

This view calls `fn_DaysOverdue()` to ensure the overdue calculation is consistent with the UDF used in the Python application.

```
CREATE VIEW View_OverdueReport AS
SELECT r.ReaderName, b.BookName, br.BorrowDate,
       fn_DaysOverdue(br.BorrowID) AS DaysOverdue
FROM Borrowing br
JOIN Readers r ON br.ReaderID = r.ReaderID
JOIN Books   b ON br.BookID   = b.BookID
WHERE br.Status = 'Borrowed'
  AND fn_DaysOverdue(br.BorrowID) > 7;
```

Listing 4.3: View_OverdueReport

4.3 Stored Procedures

4.3.1 BorrowBook(p_ReaderID, p_BookID)

```
CREATE PROCEDURE BorrowBook(IN p_ReaderID INT, IN p_BookID INT)
BEGIN
  DECLARE book_qty INT;
  SELECT Quantity INTO book_qty FROM Books WHERE BookID =
     p_BookID;
  IF book_qty > 0 THEN
    INSERT INTO Borrowing (ReaderID, BookID, BorrowDate, Status)
    VALUES (p_ReaderID, p_BookID, NOW(), 'Borrowed');
  ELSE
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Book is out of stock';
  END IF;
END
```

Listing 4.4: Stored Procedure: BorrowBook

4.3.2 ReturnBook(p_BorrowID)

```
CREATE PROCEDURE ReturnBook(IN p_BorrowID INT)
BEGIN
  UPDATE Borrowing
  SET Status = 'Returned', ReturnDate = NOW()
  WHERE BorrowID = p_BorrowID;
END
```

Listing 4.5: Stored Procedure: ReturnBook

4.4 Triggers

4.4.1 trg_reduce_quantity

Fires `AFTER INSERT` on `Borrowing` and decrements the book's `Quantity` by 1 automatically.

```
CREATE TRIGGER trg_reduce_quantity
AFTER INSERT ON Borrowing FOR EACH ROW
BEGIN
  UPDATE Books SET Quantity = Quantity - 1
  WHERE BookID = NEW.BookID;
END
```

Listing 4.6: Trigger: trg_reduce_quantity

4.4.2 trg_increase_quantity

Fires `AFTER UPDATE` on `Borrowing` and increments `Quantity` by 1 when the status changes to `'Returned'`.

```
CREATE TRIGGER trg_increase_quantity
AFTER UPDATE ON Borrowing FOR EACH ROW
BEGIN
  IF NEW.Status = 'Returned' THEN
    UPDATE Books SET Quantity = Quantity + 1
    WHERE BookID = NEW.BookID;
  END IF;
END
```

Listing 4.7: Trigger: trg_increase_quantity

4.5 User-Defined Function

4.5.1 fn_DaysOverdue(p_BorrowID)

Returns the number of days elapsed since the borrow date for any record still in `'Borrowed'` status. Returns 0 if the record has already been returned or the `BorrowID` is invalid.

```
CREATE FUNCTION fn_DaysOverdue(p_BorrowID INT)
RETURNS INT
NOT DETERMINISTIC
READS SQL DATA
```

```
BEGIN
  DECLARE borrow_dt  DATETIME;
  DECLARE days_diff  INT;
  SELECT BorrowDate INTO borrow_dt
  FROM Borrowing
  WHERE BorrowID = p_BorrowID AND Status = 'Borrowed';
  IF borrow_dt IS NULL THEN RETURN 0; END IF;
  SET days_diff = DATEDIFF(NOW(), borrow_dt);
  RETURN days_diff;
END
```

Listing 4.8: UDF: fn_DaysOverdue

Usage in `src/main.py` (function `overdue_with_udf()`):

```
SELECT r.ReaderName, b.BookName, br.BorrowDate,
       fn_DaysOverdue(br.BorrowID) AS DaysOverdue
FROM Borrowing br
JOIN Readers r ON br.ReaderID = r.ReaderID
JOIN Books   b ON br.BookID   = b.BookID
WHERE br.Status = 'Borrowed'
  AND fn_DaysOverdue(br.BorrowID) > 7
ORDER BY DaysOverdue DESC;
```

5. Python Application Development

5.1 Architecture

The application is a single-file console program (`src/main.py`) that connects to `LibraryDB` using `mysql-connector-python` and presents a hierarchical numbered menu. All output tables are formatted using the `tabulate` library with the `fancy_grid` style.

5.2 Database Connection

```
import mysql.connector
from tabulate import tabulate
from datetime import datetime

conn = mysql.connector.connect(
    host     = 'localhost',
    user     = 'root',
    password = '****',
    database = 'LibraryDB'
)
cursor = conn.cursor()
```

5.3 Menu Structure

Table 5.1: Application Menu Structure

#	Menu	Options
1	Book Management	View / Add / Edit / Delete / Search
2	Reader Management	View / Add / Update
3	Author Management	View / Add / Delete
4	Category Management	View / Add / Delete
5	Borrow / Return	Borrow Book / Return Book
6	Reports	Borrow Details / Overdue / Statistics / Top Books
7	Exit	Terminate the application

5.4 Function Reference

Table 5.2: Python Function Reference – src/main.py

Function	Description
`view_books()`	JOIN query across Books, Authors, Categories
`add_book()`	Validates author/category, confirm prompt, INSERT
`edit_book()`	UPDATE Quantity for a given BookID
`delete_book()`	Confirm name before DELETE
`search_book()`	LIKE %keyword% search with full JOIN output
`view_reader()`	SELECT * FROM Readers
`add_reader()`	INSERT with confirm; phone uniqueness enforced by DB
`update_reader()`	UPDATE Address and PhoneNumber by ReaderID
`view_authors()`	SELECT * FROM Authors
`add_author()` / `delete_author()`	INSERT / DELETE with confirm
`view_categories()`	SELECT * FROM Categories
`add_category()` / `delete_category()`	INSERT / DELETE with confirm
`borrow_book()`	Validates reader+book, calls BorrowBook proc
`return_book()`	Checks Status, shows days borrowed, calls ReturnBook
`report()`	Full borrowing history with BorrowID and Status
`overdue_with_udf()`	Calls `fn_DaysOverdue()`, filters > 7 days
`stats()`	COUNT queries: books, readers, borrow/return counts
`top_books()`	GROUP BY BookName ORDER BY COUNT DESC LIMIT 5

5.5 Key Implementation Details

5.5.1 Stored Procedure Calls

```
# Borrow a book
cursor.callproc('BorrowBook', [rid, bid])
conn.commit()

# Return a book
cursor.callproc('ReturnBook', [borrow_id])
conn.commit()
```

5.5.2 Overdue Detection in CLI

The `return_book()` function computes days borrowed locally and warns the user before confirming the return:

```
from datetime import datetime

borrow_date = data[3]           # DATETIME from DB
days = (datetime.now() - borrow_date).days
if days > 7:
    print('OVERDUE! Late return!')
```

5.5.3 Duplicate Return Prevention

```
if data[4] == 'Returned':
    print('This book has already been returned!')
    return
```

5.5.4 Error Handling

```
try:
    cursor.callproc('BorrowBook', [rid, bid])
    conn.commit()
    cursor.execute('SELECT MAX(BorrowID) FROM Borrowing')
    borrow_id = cursor.fetchone()[0]
    print(f'Borrow successful! Borrow ID = {borrow_id}')
except Exception as e:
    print('Error:', e)
```

6. Database Security & Administration

6.1 User Management & Permissions

Table 6.1: Proposed Database Roles

User	Host	Privileges
app_user	localhost	SELECT, INSERT, UPDATE on all tables; EXECUTE on stored procedures
report_user	localhost	SELECT only on View_BookDetails, View_BorrowingReport, View_OverdueReport
admin_user	localhost	ALL PRIVILEGES on LibraryDB (DDL + DML)

```
CREATE USER 'app_user'@'localhost' IDENTIFIED BY '
    secure_password';
GRANT SELECT, INSERT, UPDATE ON LibraryDB.* TO 'app_user'@'
    localhost';
GRANT EXECUTE ON PROCEDURE LibraryDB.BorrowBook TO 'app_user'@'
    localhost';
GRANT EXECUTE ON PROCEDURE LibraryDB.ReturnBook TO 'app_user'@'
    localhost';
FLUSH PRIVILEGES;
```

6.2 Data Security Measures

- **SQL Injection Prevention:** All queries in `src/main.py` use parameterized statements (`%s` placeholders), never string concatenation.
- **Domain Integrity:** CHECK constraints enforce `Quantity >= 0` and `Status IN ('Borrowed','Returned')`.
- **Referential Integrity:** `ON DELETE RESTRICT` prevents deletion of authors or categories still referenced by books.
- **Uniqueness:** UNIQUE constraints on `CategoryName` and `PhoneNumber` prevent duplicate entries.
- **Credentials:** For this academic project, credentials are stored in source code.

Production systems should use environment variables or a secrets manager.

6.3 Backup & Recovery Plan

- Daily full logical backups using `mysqldump`, stored with timestamp.
- Binary logging enabled for incremental backup and point-in-time recovery.
- Backup files stored off-site or in cloud object storage (e.g., AWS S3).
- Monthly restore drills to verify backup integrity and recovery time.

```
mysqldump -u root -p LibraryDB > LibraryDB_$(date +%Y%m%d).sql
```

Listing 6.1: Daily backup command

6.4 Performance Optimization

- Indexes on `BookName` and `AuthorID` reduce full-table scans.
- Views pre-join commonly accessed multi-table data, reducing query complexity.
- Stored procedures execute multi-step logic server-side, minimizing round trips.
- Single persistent connection avoids repeated connection overhead.

7. System Demonstration

7.1 SQL Script Execution (MySQL Workbench)

```
49 •  SHOW TABLES;
50 •  DESCRIBE Books;
51
52 •  INSERT INTO Categories (CategoryNa
```

100% 13:49

Result Grid Filter Rows: Search E

Tables_in_librarydb
Authors
Books
Borrowing
Categories
Readers
view_bookdetails
view_borrowingreport
view_overduereport

Figure 7.1: SHOW TABLES – confirms all 5 tables exist in LibraryDB

```
50 •  DESCRIBE Books;
51
52 •  INSERT INTO Categories (CategoryName) VALUES
```

100% 1:50

Result Grid Filter Rows: Search Export:

Field	Type	Null	Key	Default	Extra
BookID	int	NO	PRI	NULL	auto_increment
BookName	varchar(255)	NO	MUL	NULL	
AuthorID	int	NO	MUL	NULL	
PublishYear	int	YES		NULL	
Quantity	int	NO		0	
CategoryID	int	NO	MUL	NULL	

Figure 7.2: DESCRIBE Books – column definitions, data types, nullability, key status

```
112 •  SELECT * FROM Books;
113 •  SELECT * FROM Readers;
```

100% 21:112

Result Grid Filter Rows: Search Edit: Export/

BookID	BookName	AuthorID	PublishYear	Quantity	CategoryID
1	Mat Biec	1	1990	14	1
2	Harry Potter	2	1997	11	1
3	A Brief History of Time	3	1988	8	2
4	Sapiens	4	2011	12	3
5	The Art of Computer Programming	5	1968	5	4
6	How to Win Friends	6	1936	20	5
7	Rich Dad Poor Dad	7	1997	18	7
8	The Alchemist	8	1988	14	1
9	Murder on the Orient Express	9	1934	9	1
10	The Da Vinci Code	10	2003	11	1
NULL	NULL	NULL	NULL	NULL	NULL

Figure 7.3: SELECT * FROM Books – all 10 book records with full field data

```
113 •  SELECT * FROM Readers;
114 •  SELECT * FROM Borrow
115
```

100% 23:113

Result Grid Filter Rows: Search

ReaderID	ReaderName	Address	PhoneNumber
1	Tran Van A	Hanoi	0900000001
2	Le Thi B	HCM	0900000002
3	Pham Van C	Danang	0900000003
4	Nguyen Thi D	Hue	0900000004
5	Hoang Van E	Can Tho	0900000005
6	Bui Van F	Hanoi	0900000006
7	Do Thi G	HCM	0900000007
8	Vu Van H	Haiphong	0900000008
9	Phan Thi I	Quang Ninh	0900000009
10	Dang Van K	Nghe An	0900000010
11	Vi Anh	Ha Noi	09xx
12	io	kk	i
13	fgh	f	4
14	Huy	HCM	0922

Figure 7.4: SELECT * FROM Readers – all 10 reader entries

```
114 •  SELECT * FROM Borrowing;
115
116 •  CREATE INDEX idx_bookname ON Books(BookName);
117 •  CREATE INDEX idx_author ON Books(AuthorID);
```

100% 25:114

Result Grid Filter Rows: Search Edit:

BorrowID	ReaderID	BookID	BorrowDate	ReturnDate	Status
1	1	1	2026-04-18 15:33:23	2026-04-27 23:17:49	Returned
2	2	2	2026-04-27 17:56:57	NULL	Borrowed
3	3	3	2026-04-27 17:56:57	2026-04-27 17:56:57	Returned
4	4	4	2026-04-13 15:33:23	NULL	Borrowed
5	5	5	2026-04-27 17:56:57	2026-04-27 17:56:57	Returned
6	6	6	2026-04-20 15:33:23	NULL	Borrowed
7	7	7	2026-04-27 17:56:57	2026-04-27 17:56:57	Returned
8	8	8	2026-04-27 17:56:57	NULL	Borrowed
9	9	9	2026-04-27 17:56:57	2026-04-27 17:56:57	Returned
10	10	10	2026-04-27 17:56:57	NULL	Borrowed
11	1	1	2026-04-27 17:56:57	NULL	Borrowed
12	2	1	2026-04-27 17:56:57	2026-04-28 01:22:23	Returned
13	1	2	2026-04-27 17:59:34	NULL	Borrowed
14	1	2	2026-04-27 23:17:28	NULL	Borrowed
15	1	2	2026-04-28 01:22:15	NULL	Borrowed
16	1	10	2026-04-28 01:28:29	2026-04-28 01:28:40	Returned
17	12	2	2026-04-28 15:18:51	NULL	Borrowed
18	10	2	2026-04-28 15:29:22	2026-04-28 15:29:32	Returned

Figure 7.5: SELECT * FROM Borrowing – all 10 borrowing transactions with Status

```
130 •  SELECT * FROM View_BookDetails;
131
132 •  CREATE VIEW View_BorrowingReport AS
133    SELECT
```

100% 32:130

Result Grid Filter Rows: Search Export:

BookID	BookName	AuthorName	CategoryName	Quantity
1	Mat Biec	Nguyen Nhat Anh	Novel	14
2	Harry Potter	J.K. Rowling	Novel	11
3	A Brief History of Time	Stephen Hawking	Science	8
4	Sapiens	Yuval Noah Harari	History	12
5	The Art of Computer Programming	Donald Knuth	Technology	5
6	How to Win Friends	Dale Carnegie	Education	20
7	Rich Dad Poor Dad	Robert Kiyosaki	Business	18
8	The Alchemist	Paulo Coelho	Novel	14
9	Murder on the Orient Express	Agatha Christie	Novel	9
10	The Da Vinci Code	Dan Brown	Novel	11

Figure 7.6: SELECT * FROM View_BookDetails – catalog view with joined author/-category names

```
143 •  SELECT * FROM View_BorrowingReport;
144    DELIMITER //
145
```

100% 36:143

Result Grid Filter Rows: Search Export:

ReaderName	BookName	BorrowDate	ReturnDate	Status
Pham Van C	A Brief History of Time	2026-04-27 17:56:57	2026-04-27 17:56:57	Returned
Le Thi B	Harry Potter	2026-04-27 17:56:57	NULL	Borrowed
Tran Van A	Harry Potter	2026-04-27 17:59:34	NULL	Borrowed
Tran Van A	Harry Potter	2026-04-27 23:17:28	NULL	Borrowed
Tran Van A	Harry Potter	2026-04-28 01:22:15	NULL	Borrowed
io	Harry Potter	2026-04-28 15:18:51	NULL	Borrowed
Dang Van K	Harry Potter	2026-04-28 15:29:22	2026-04-28 15:29:32	Returned
Bui Van F	How to Win Friends	2026-04-20 15:33:23	NULL	Borrowed
Tran Van A	Mat Biec	2026-04-18 15:33:23	2026-04-27 23:17:49	Returned
Tran Van A	Mat Biec	2026-04-27 17:56:57	NULL	Borrowed
Le Thi B	Mat Biec	2026-04-27 17:56:57	2026-04-28 01:22:23	Returned
Phan Thi I	Murder on the Orient...	2026-04-27 17:56:57	2026-04-27 17:56:57	Returned
Do Thi G	Rich Dad Poor Dad	2026-04-27 17:56:57	2026-04-27 17:56:57	Returned
Nguyen Thi D	Sapiens	2026-04-13 15:33:23	NULL	Borrowed
Vu Van H	The Alchemist	2026-04-27 17:56:57	NULL	Borrowed
Hoang Van E	The Art of Computer...	2026-04-27 17:56:57	2026-04-27 17:56:57	Returned
Dang Van K	The Da Vinci Code	2026-04-27 17:56:57	NULL	Borrowed
Tran Van A	The Da Vinci Code	2026-04-28 01:28:29	2026-04-28 01:28:40	Returned

Figure 7.7: SELECT * FROM View_BorrowingReport – full transaction history view

```
217 •  SHOW INDEX FROM Books;
218
219    DELIMITER //
```

100% 23:217

Result Grid Filter Rows: Search Export:

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
Books	0	PRIMARY	1	BookID	A	10	NULL	NULL		BTREE			YES	NULL
Books	1	CategoryID	1	CategoryID	A	6	NULL	NULL		BTREE			YES	NULL
Books	1	idx_bookname	1	BookName	A	10	NULL	NULL		BTREE			YES	NULL
Books	1	idx_author	1	AuthorID	A	10	NULL	NULL		BTREE			YES	NULL

Figure 7.8: SHOW INDEX FROM Books – confirms idx_bookname and idx_author are active

7.2 Python CLI Application (src/main.py)

```
======== LIBRARY SYSTEM ========
1. Book Management
2. Reader Management
3. Author Management
4. Category Management
5. Borrow / Return
6. Reports
7. Exit
Choose: 1

--- BOOK MANAGEMENT ---
1. View
2. Add
3. Edit
4. Delete
5. Search
6. Back
Choose: 1
```

ID	Name	Author	Year	Qty	Category
1	Mat Biec	1 – Nguyen Nhat Anh	1990	14	1 – Novel
2	Harry Potter	2 – J.K. Rowling	1997	11	1 – Novel
3	A Brief History of Time	3 – Stephen Hawking	1988	8	2 – Science
4	Sapiens	4 – Yuval Noah Harari	2011	12	3 – History
5	The Art of Computer Programming	5 – Donald Knuth	1968	5	4 – Technology
6	How to Win Friends	6 – Dale Carnegie	1936	20	5 – Education
7	Rich Dad Poor Dad	7 – Robert Kiyosaki	1997	18	7 – Business
8	The Alchemist	8 – Paulo Coelho	1988	14	1 – Novel
9	Murder on the Orient Express	9 – Agatha Christie	1934	9	1 – Novel
10	The Da Vinci Code	10 – Dan Brown	2003	11	1 – Novel

Figure 7.9: Application start – Connected to LibraryDB and book list

```python
import mysql.connector
from tabulate import tabulate

# ===== CONNECT =====
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="1511",
    database="LibraryDB"
)
cursor = conn.cursor()

print("✅ Connected to LibraryDB!")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                  Python

/usr/local/bin/python3 "/Users/vianhbui/Downloads/Final Database/test2.py"
(base) vianhbui@MacBook-Air-cua-Vi-Anh Final Database % /usr/local/bin/python3 "/Users/vianhbui/Downloads/Final Database/test2.py"
✅ Connected to LibraryDB!

========= LIBRARY SYSTEM =========
1. Book Management
2. Reader Management
3. Author Management
4. Category Management
5. Borrow / Return
6. Reports
7. Exit
Choose:
```

Figure 7.10: Main menu – all 7 options displayed

```
--- BOOK MANAGEMENT ---
1. View
2. Add
3. Edit
4. Delete
5. Search
6. Back
Choose: 2
Book name: Conan
Author ID: 1
Year: 2020
Quantity: 10
Category ID: 1

===== CONFIRM ADD BOOK =====
Name: Conan
Author: 1 – Nguyen Nhat Anh
Year: 2020
Quantity: 10
Category: 1 – Novel
Confirm? (y/n): y
✅ Book added! ID = 18
```

Figure 7.11: Book Management → Add – confirm prompt with author/category lookup

```
--- BOOK MANAGEMENT ---
1. View
2. Add
3. Edit
4. Delete
5. Search
6. Back
Choose: 3
Book ID: 18
New quantity: 21
✅ Book updated!
```

Figure 7.12: Book Management → Edit – update quantity for a book

```
--- BOOK MANAGEMENT ---
1. View
2. Add
3. Edit
4. Delete
5. Search
6. Back
Choose: 4
Book ID: 18

===== CONFIRM DELETE BOOK =====
Book: Conan (ID: 18)
Confirm? (y/n): y
✅ Deleted!
```

Figure 7.13: Book Management → Delete – confirm prompt showing book name

```
--- BOOK MANAGEMENT ---
1. View
2. Add
3. Edit
4. Delete
5. Search
6. Back
Choose: 5
Search name: co
```

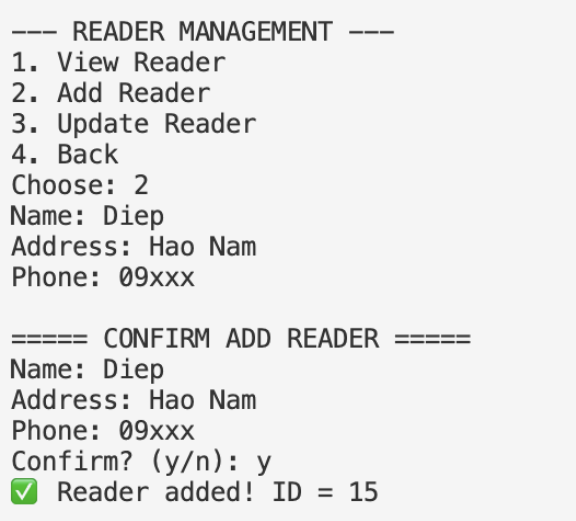
ID	Name	Author	Year	Qty	Category
5	The Art of Computer Programming	5 - Donald Knuth	1968	5	4 - Technology
10	The Da Vinci Code	10 - Dan Brown	2003	11	1 - Novel

Figure 7.14: Book Management → Search – keyword search returning matching results

```
--- READER MANAGEMENT ---
1. View Reader
2. Add Reader
3. Update Reader
4. Back
Choose: 1
```

ID	Name	Address	Phone
1	Tran Van A	Hanoi	0900000001
2	Le Thi B	HCM	0900000002
3	Pham Van C	Danang	0900000003
4	Nguyen Thi D	Hue	0900000004
5	Hoang Van E	Can Tho	0900000005
6	Bui Van F	Hanoi	0900000006
7	Do Thi G	HCM	0900000007
8	Vu Van H	Haiphong	0900000008
9	Phan Thi I	Quang Ninh	0900000009
10	Dang Van K	Nghe An	0900000010
11	Vi Anh	Ha Noi	09xx
12	io	kk	i
13	fgh	f	4
14	Huy	HCM	0922

Figure 7.15: Reader Management → View – full reader list



```
--- READER MANAGEMENT ---
1. View Reader
2. Add Reader
3. Update Reader
4. Back
Choose: 2
Name: Diep
Address: Hao Nam
Phone: 09xxx

===== CONFIRM ADD READER =====
Name: Diep
Address: Hao Nam
Phone: 09xxx
Confirm? (y/n): y
✅ Reader added! ID = 15
```

Figure 7.16: Reader Management → Add Reader – confirm prompt, success with new ID

```
--- READER MANAGEMENT ---
1. View Reader
2. Add Reader
3. Update Reader
4. Back
Choose: 3
Reader ID: 15
New address: Vu Thanh
New phone: 09xxx
✅ Reader updated!

--- READER MANAGEMENT ---
1. View Reader
2. Add Reader
3. Update Reader
4. Back
Choose: 1
```

ID	Name	Address	Phone
1	Tran Van A	Hanoi	0900000001
2	Le Thi B	HCM	0900000002
3	Pham Van C	Danang	0900000003
4	Nguyen Thi D	Hue	0900000004
5	Hoang Van E	Can Tho	0900000005
6	Bui Van F	Hanoi	0900000006
7	Do Thi G	HCM	0900000007
8	Vu Van H	Haiphong	0900000008
9	Phan Thi I	Quang Ninh	0900000009
10	Dang Van K	Nghe An	0900000010
11	Vi Anh	Ha Noi	09xx
12	io	kk	i
13	fgh	f	4
14	Huy	HCM	0922
15	Diep	Vu Thanh	09xxx

Figure 7.17: Reader Management → Update Reader – updated address and phone

```
--- AUTHOR MANAGEMENT ---
1. View Authors
2. Add Author
3. Delete Author
4. Back
Choose: 1
```

ID	Author
1	Nguyen Nhat Anh
2	J.K. Rowling
3	Stephen Hawking
4	Yuval Noah Harari
5	Donald Knuth
6	Dale Carnegie
7	Robert Kiyosaki
8	Paulo Coelho
9	Agatha Christie
10	Dan Brown

```
--- AUTHOR MANAGEMENT ---
1. View Authors
2. Add Author
3. Delete Author
4. Back
Choose: 2
Author name: Vi Anh

===== CONFIRM ADD AUTHOR =====
Author: Vi Anh
Confirm? (y/n): y
✅ Author added! ID = 12
```

Figure 7.18: Author Management → View + Add Author

```
--- CATEGORY MANAGEMENT ---
1. View Categories
2. Add Category
3. Delete Category
4. Back
Choose: 1
```

ID	Category
8	Art
7	Business
9	Children
5	Education
3	History
1	Novel
10	Philosophy
6	Psychology
2	Science
4	Technology

Figure 7.19: Category Management → View – full category list

```
--- CATEGORY MANAGEMENT ---
1. View Categories
2. Add Category
3. Delete Category
4. Back
Choose: 2
Category name: Sci-Fi

===== CONFIRM ADD CATEGORY =====
Category: Sci-Fi
Confirm? (y/n): y
✅ Category added! ID = 11

--- CATEGORY MANAGEMENT ---
1. View Categories
2. Add Category
3. Delete Category
4. Back
Choose: 3
Category ID: 11

===== CONFIRM DELETE CATEGORY =====
Category: Sci-Fi (ID: 11)
Confirm? (y/n): y
✅ Deleted!
```

Figure 7.20: Category Management → Add + Delete Category

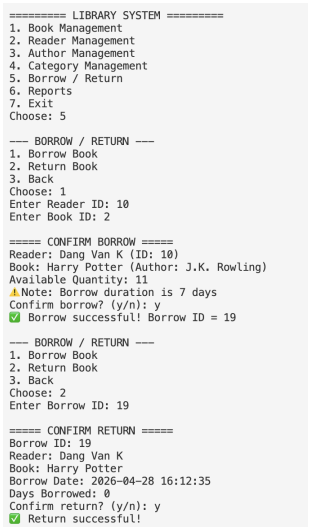


Figure 7.21: Borrow / Return → Borrow Book – confirm screen, borrow successful

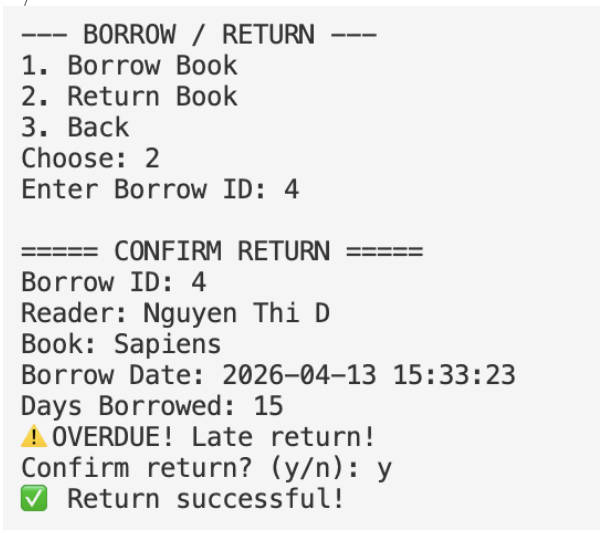
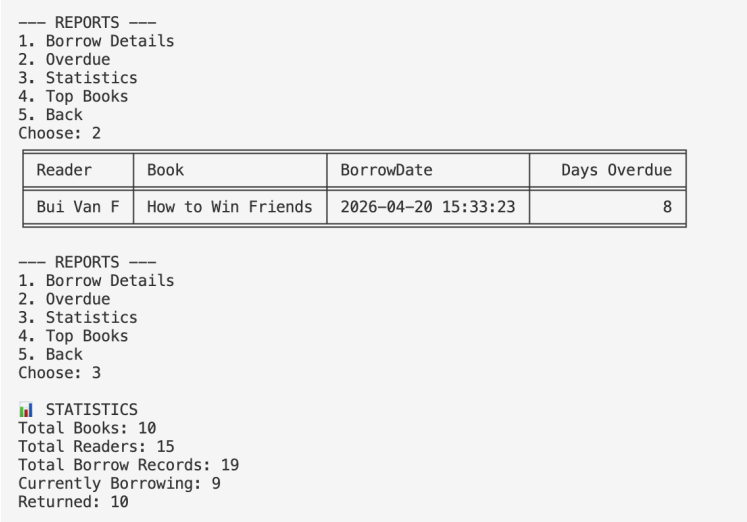


Figure 7.22: Borrow / Return → Return Overdue Book – overdue warning (days > 7)

```
--- REPORTS ---
1. Borrow Details
2. Overdue
3. Statistics
4. Top Books
5. Back
Choose: 1
```

BorrowID	Reader	Book	BorrowDate	ReturnDate	Status
3	Pham Van C	A Brief History of Time	2026-04-27 17:56:57	2026-04-27 17:56:57	Returned
2	Le Thi B	Harry Potter	2026-04-27 17:56:57		Borrowed
13	Tran Van A	Harry Potter	2026-04-27 17:59:34		Borrowed
14	Tran Van A	Harry Potter	2026-04-27 23:17:28		Borrowed
15	Tran Van A	Harry Potter	2026-04-28 01:22:15		Borrowed
17	io	Harry Potter	2026-04-28 15:18:51		Borrowed
18	Dang Van K	Harry Potter	2026-04-28 15:29:22	2026-04-28 15:29:32	Returned
19	Dang Van K	Harry Potter	2026-04-28 16:12:35	2026-04-28 16:12:44	Returned
6	Bui Van F	How to Win Friends	2026-04-20 15:33:23		Borrowed
1	Tran Van A	Mat Biec	2026-04-18 15:33:23	2026-04-27 23:17:49	Returned
11	Tran Van A	Mat Biec	2026-04-27 17:56:57		Borrowed
12	Le Thi B	Mat Biec	2026-04-27 17:56:57	2026-04-28 01:22:23	Returned
9	Phan Thi I	Murder on the Orient Express	2026-04-27 17:56:57	2026-04-27 17:56:57	Returned
7	Do Thi G	Rich Dad Poor Dad	2026-04-27 17:56:57	2026-04-27 17:56:57	Returned
4	Nguyen Thi D	Sapiens	2026-04-13 15:33:23	2026-04-28 16:14:20	Returned
8	Vu Van H	The Alchemist	2026-04-27 17:56:57		Borrowed
5	Hoang Van E	The Art of Computer Programming	2026-04-27 17:56:57	2026-04-27 17:56:57	Returned
10	Dang Van K	The Da Vinci Code	2026-04-27 17:56:57		Borrowed
16	Tran Van A	The Da Vinci Code	2026-04-28 01:28:29	2026-04-28 01:28:40	Returned

Figure 7.23: Reports → Borrow Details – full borrowing history table



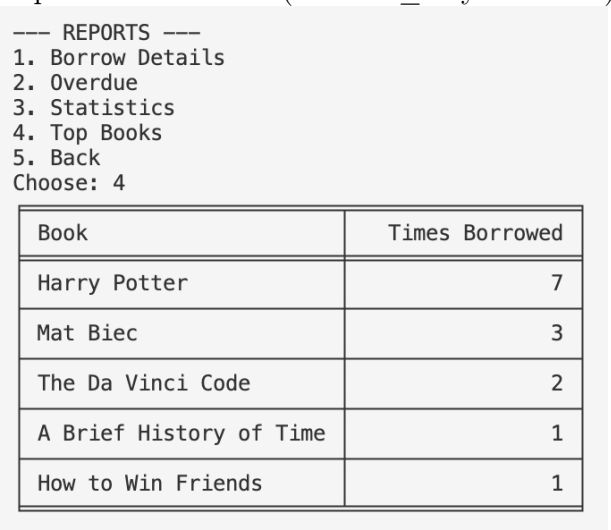
```
--- REPORTS ---
1. Borrow Details
2. Overdue
3. Statistics
4. Top Books
5. Back
Choose: 2
```

Reader	Book	BorrowDate	Days Overdue
Bui Van F	How to Win Friends	2026-04-20 15:33:23	8

```
--- REPORTS ---
1. Borrow Details
2. Overdue
3. Statistics
4. Top Books
5. Back
Choose: 3

📊 STATISTICS
Total Books: 10
Total Readers: 15
Total Borrow Records: 19
Currently Borrowing: 9
Returned: 10
```

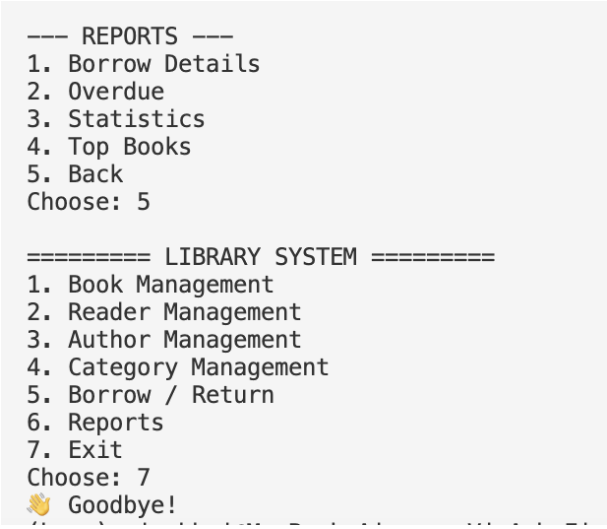
Figure 7.24: Reports → Overdue (UDF fn_DaysOverdue) + Statistics



```
--- REPORTS ---
1. Borrow Details
2. Overdue
3. Statistics
4. Top Books
5. Back
Choose: 4
```

Book	Times Borrowed
Harry Potter	7
Mat Biec	3
The Da Vinci Code	2
A Brief History of Time	1
How to Win Friends	1

Figure 7.25: Reports → Top Books – top 5 most borrowed titles



```
--- REPORTS ---
1. Borrow Details
2. Overdue
3. Statistics
4. Top Books
5. Back
Choose: 5

========== LIBRARY SYSTEM ==========
1. Book Management
2. Reader Management
3. Author Management
4. Category Management
5. Borrow / Return
6. Reports
7. Exit
Choose: 7
👋 Goodbye!
```

Figure 7.26: Application exit – goodbye message and successful termination

8. Conclusion & Recommendations

8.1 Project Outcome Evaluation

The Library Management System successfully fulfills all requirements specified in the project brief. The database schema is fully normalized with comprehensive integrity enforcement, and all required advanced database objects have been implemented and demonstrated. The Python application (`src/main.py`) provides a clean and functional interface for every core library operation.

Table 8.1: Requirements Completion Summary

Requirement	**Status**	**Notes**
5 normalized tables with FK constraints	Done	All constraints active
10 sample records per table	Done	Plus overdue test data
Indexes (2)	Done	idx_bookname, idx_author
Views (3)	Done	BookDetails, BorrowingReport, OverdueReport
Stored Procedures (2)	Done	BorrowBook, ReturnBook
Triggers (2)	Done	Quantity auto-management
User-Defined Function	Done	fn_DaysOverdue
Python CRUD Application	Done	6 menu modules, 18+ functions
Overdue Alert System	Done	CLI warning + UDF report
Statistical Reports	Done	Stats, Top Books, Full Report

8.2 Limitations

Despite achieving the main objectives, the system still has several limitations:

1. The application is implemented as a command-line interface (CLI), which may

not provide an optimal user experience compared to modern graphical or web-based systems.

2. The system does not include an authentication or authorization mechanism. All users are assumed to have equal access privileges, which is not suitable for real-world deployment where role-based access control is required.
3. Concurrency control and transaction conflicts have not been thoroughly tested. In a multi-user environment, simultaneous borrowing or returning actions may lead to inconsistencies without proper isolation handling.
4. The system does not support advanced features such as book reservation, fine calculation for overdue returns, or real-time notifications. These features are important in practical library systems and could be considered for future development.

8.3 Recommendations for Future Improvements

1. Add a `DueDate` column to `Borrowing` and implement an automatic fine calculation function based on days overdue.
2. Implement full-text search using a `FULLTEXT` index on `Books(BookName)` for more flexible catalog search.
3. Migrate `src/main.py` to a web-based interface (e.g., Flask + HTML) or desktop GUI (e.g., Tkinter).
4. Add role-based authentication at the application level, mapping login credentials to MySQL user roles (librarian vs. administrator).
5. Implement an audit log table populated by triggers to track all DML operations with timestamps and user context.
6. Add a reservation system allowing readers to reserve out-of-stock books.

References

[1] MySQL 8.0 Reference Manual. Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en/

[2] mysql-connector-python Developer Guide. Oracle Corporation. https://dev.mysql.com/doc/connector-python/en/

[3] tabulate Python Library Documentation. https://pypi.org/project/tabulate/

[4] Forta, B. (2013). *MySQL Crash Course*. Sams Publishing.

[5] Hoffer, J., Ramesh, V., & Topi, H. (2016). *Modern Database Management*, 12th Edition. Pearson.

[6] Date, C. J. (2003). *An Introduction to Database Systems*, 8th Edition. Addison-Wesley.